UNIVERSIDAD DEL VALLE DE GUATEMALA

Facultad de Ingeniería
Computación Paralela
Semestre II – 2026



Actividad en clase

Intersección Semáforo Burger King Zona 5

Integrante:
Nájera Marakovits, Ingrid Nina Alessandra – 231088

1. Objetivo

El objetivo de la actividad fue construir una simulación sencilla de una intersección controlada por semáforos y utilizar OpenMP para estudiar distintas estrategias de ejecución. La simulación utiliza estructuras para representar vehículos, semáforos e intersecciones; actualiza periódicamente los semáforos y permite que los vehículos avancen cuando el semáforo asociado está en verde.

Además de comprobar el funcionamiento de la simulación, se compararon tres estrategias de ejecución: secuencial, paralela con planificación estática y paralela con planificación dinámica.

2. Estrategias probadas

2.1. Ejecución secuencial

En la versión secuencial no se utilizan directivas de OpenMP para repartir los ciclos de trabajo. Cada semáforo y cada vehículo se procesa uno después del otro mediante ciclos `for`. Esta versión funciona como referencia para comparar el costo adicional introducido por el paralelismo.

```
for (int i = 0; i < num_vehicles; i++) {
    int light_index = i % num_lights;
    if (lights[light_index].state == 1) {
        vehicles[i].position++;
    }
}
```

2.2. Paralelismo estático

En la estrategia estática se utilizó `schedule(static)`. OpenMP divide las iteraciones entre los hilos antes de iniciar el ciclo paralelo. Esta estrategia tiene poco costo de administración y suele funcionar bien cuando todas las iteraciones requieren una cantidad de trabajo parecida.

```
#pragma omp parallel for schedule(static)
for (int i = 0; i < num_vehicles; i++) {
    int light_index = i % num_lights;
    if (lights[light_index].state == 1) {
        vehicles[i].position++;
    }
}
```

2.3. Paralelismo dinámico

En la estrategia dinámica se utilizó `schedule(dynamic)`. En este caso los hilos reciben trabajo conforme van terminando iteraciones anteriores. Esto puede ayudar cuando la carga es irregular, aunque introduce un costo adicional de planificación.

```
#pragma omp parallel for schedule(dynamic)
for (int i = 0; i < num_vehicles; i++) {
    int light_index = i % num_lights;
    if (lights[light_index].state == 1) {
        vehicles[i].position++;
    }
}
```

También se trabajó con ajuste dinámico del número de hilos durante la simulación mediante `omp_set_dynamic(1)` y `omp_set_num_threads()`. Es importante distinguir este mecanismo de `schedule(dynamic)`: el primero se relaciona con la cantidad de hilos que puede utilizar el equipo, mientras el segundo controla cómo se reparten las iteraciones del ciclo.

3. Metodología de prueba

Para comparar las estrategias se ejecutaron configuraciones con 10, 50 y 100 vehículos. Cada prueba realizó 10,000 iteraciones. Antes de ejecutar cada modo se reinicializaron los vehículos y semáforos utilizando la misma semilla, con el objetivo de partir de condiciones comparables.

En las pruebas paralelas se utilizaron cuatro hilos y se deshabilitó el ajuste automático del número de hilos durante la medición, de forma que la diferencia principal entre las pruebas paralelas fuera el tipo de planificación: `static` o `dynamic`. El tiempo se midió con `omp_get_wtime()`.

No se incluyó la espera de un segundo de la simulación normal dentro de las pruebas de rendimiento, ya que esa pausa no representa trabajo computacional y distorsionaría la medición.

4. Resultados

Los resultados obtenidos en la ejecución mostrada en clase fueron los siguientes:

Cuadro 1: Tiempos observados para las tres estrategias.

Vehículos	Secuencial (s)	Static (s)	Dynamic (s)
10	0.000000	0.506000	0.501000
50	0.000000	0.516000	0.499000
100	0.002000	0.532000	0.504000

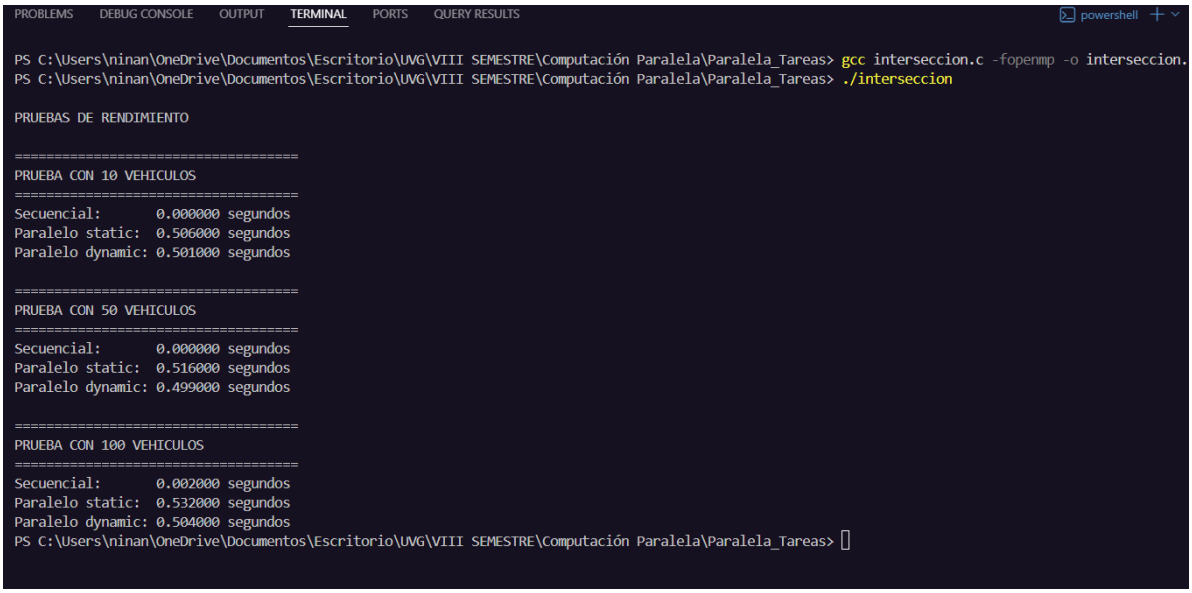


Figura 1: Salida de las pruebas de rendimiento ejecutadas en la terminal.

5. Análisis de resultados

En estas pruebas la ejecución secuencial fue considerablemente más rápida que las dos variantes paralelas. Esto ocurre porque la carga de trabajo por iteración es muy pequeña: únicamente se actualizan cuatro semáforos y entre 10 y 100 vehículos. Para un problema de esta granularidad, el costo de crear y sincronizar regiones paralelas puede ser mayor que el trabajo útil realizado dentro de ellas.

Los valores de 0.000000 segundos observados para 10 y 50 vehículos indican que el tiempo secuencial fue demasiado pequeño para apreciarse con seis cifras decimales en esa ejecución. Con 100 vehículos se registraron 0.002000 segundos, mientras que las versiones paralelas permanecieron alrededor de medio segundo.

Entre las dos estrategias paralelas, la planificación dinámica obtuvo tiempos ligeramente menores en las tres configuraciones: 0.501 frente a 0.506 segundos para 10 vehículos, 0.499 frente a 0.516 segundos para 50 y 0.504 frente a 0.532 segundos para 100. Sin embargo, la diferencia entre ambas es pequeña comparada con el costo total del paralelismo.

Estos resultados no significan que el paralelismo sea siempre más lento. Lo que muestran es que el tamaño del trabajo debe ser suficientemente grande para compensar el costo de crear hilos, distribuir iteraciones y sincronizar regiones. Una posible optimización sería aumentar la carga de trabajo por región paralela o mantener una región paralela activa durante más operaciones, evitando crear equipos de hilos repetidamente en cada iteración.

6. Conclusiones

- La versión secuencial fue la más rápida para las configuraciones evaluadas debido al bajo volumen de trabajo.
- `schedule(static)` realiza un reparto previo y simple de iteraciones, mientras `schedule(dynamic)` redistribuye trabajo conforme los hilos terminan.
- En las pruebas realizadas, `schedule(dynamic)` fue ligeramente más rápido que `schedule(static)`, aunque ambos quedaron muy por detrás de la versión secuencial.
- El experimento demuestra que paralelizar no garantiza automáticamente una mejora de rendimiento; la granularidad del problema y el overhead de OpenMP son factores determinantes.
- Para cargas mayores o simulaciones más complejas, el uso de OpenMP puede resultar más beneficioso al existir suficiente trabajo para repartir entre los hilos.